

Pydantic

■ Pydantic

- Python 타입 힌트를 이용해 데이터 검증(validation)과 직렬화(serialization)를 처리하는 라이브러리
- FastAPI 내부에서 요청 본문(request body) 검증에 사용

```
from pydantic import BaseModel

class User(BaseModel):
    id: int
    name: str
    email: str
    age: int | None = None # 선택적 필드 (기본값 None)

user = User(id=1, name="홍길동", email="hong@test.com")
print(user.id)          # 1
print(user.model_dump()) # dict로 변환
print(user.model_dump_json()) # JSON 문자열로 변환

User(id="not_a_number", name="hong", email="x")
```

■ pydantic

- 데이터 모델 정의 및 유효성 검사 라이브러리

기능	설명
Field()	필드에 제약조건/메타데이터 추가 age: int = Field(gt=0, le=150)
Validator	커스텀 검증 로직 @field_validator("email")
Nested Model	모델 안에 모델 포함 address: Address
Config / model_config	모델 동작 설정 (예: 별칭 허용 등) model_config = ConfigDict(...)
Optional/Union	선택적 필드, 여러 타입 허용 value: int str

Pydantic

■ Pydantic

- FastAPI는 Pydantic을 "엔진"처럼 내장해서 요청 body 검증, 응답 스키마 생성, 자동 문서화(OpenAPI)까지 처리합니다.

```
from pydantic import BaseModel, Field, field_validator

class Product(BaseModel):
    name: str = Field(min_length=1, max_length=50)
    price: float = Field(gt=0, description="상품 가격")

    @field_validator("name")
    @classmethod
    def name_must_not_contain_space(cls, v):
        if " " in v:
            raise ValueError("이름에 공백이 들어갈 수 없습니다")
        return v
```

FastAPI

■ FastAPI

- Python으로 웹 서버/라우팅/요청-응답 처리 프레임워크

특징	설명
고성능	Starlette(비동기 웹 프레임워크) 기반, Node.js/Go급 성능
자동 문서화	Swagger UI(/docs), ReDoc(/redoc) 자동 생성
타입 힌트 기반	Python 타입 어노테이션만으로 유효성 검사, 직렬화 자동 처리
비동기 지원	async def 으로 비동기 처리 네이티브 지원
Pydantic	요청/응답 데이터 검증을 Pydantic이 전담

FastAPI

■ FastAPI

- Python으로 웹 API를 만들기 위한 프레임워크

```
from fastapi import FastAPI

app = FastAPI()

class Item(BaseModel):
    name: str
    price: float
    is_offer: bool | None = None

@app.get("/")
def read_root():
    return {"message": "Hello World"}

@app.get("/items/{item_id}")
def read_item(item_id: int, q: str | None = None):    #FastAPI가 자동으로 형변환 + 검증
    return {"item_id": item_id, "q": q}
#클라이언트가 JSON body를 보내면 FastAPI가 자동으로 Item 모델로 파싱 + 검증
@app.post("/items/")
def create_item(item: Item):
    return {"item_name": item.name, "item_price": item.price}
```

FastAPI

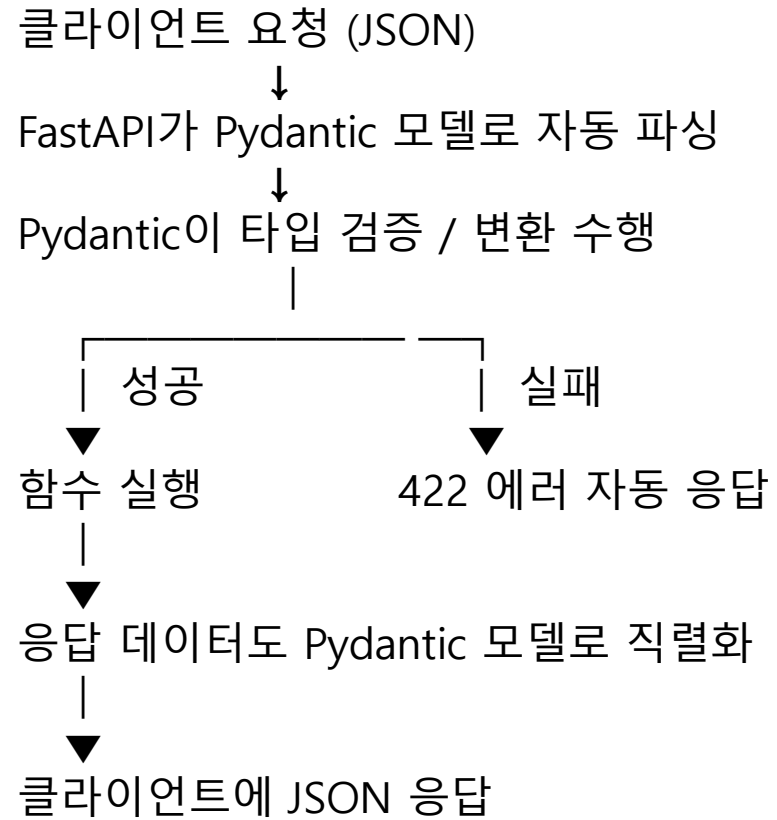
■ FastAPI

■ 비동기 처리

```
@app.get("/async-data")
async def get_data():
    result = await some_async_db_call()
    return result
```

FastAPI

■ FastAPI + Pydantic 전체 흐름



■ Uvicorn

- Python의 ASGI(Asynchronous Server Gateway Interface) 웹 서버
- FastAPI, Starlette 등의 웹 애플리케이션을 실행하는 고성능 서버
- asyncio 기반
- uvloop 사용 가능
- httptools 사용 가능
- WebSocket 지원
- HTTP Keep-Alive 지원
- Hot Reload 지원

```
pip install uvicorn
```

WSGI : 동기(Synchronous) 처리 , 요청 하나를 처리하는 동안 다른 요청은 대기
ASGI : 비동기(Asynchronous) 지원, WebSocket 지원, HTTP/2 지원, 높은 동시성

FastAPI

■ Uvicorn

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def home():
    return {"message": "Hello Uvicorn"}
```

uvicorn Python 파일명:FastAPI 객체

uvicorn Python 파일명:FastAPI 객체 --reload
파일 변경 감지
자동 재시작
개발용

LangChain & Message

LangChain 이란

■ LangChain

- 해리슨 체이스(Harrison Chase)에 의해 2022년 10월에 오픈 소스 프로젝트로 시작
- 대규모 언어 모델(LLM)을 활용하여 애플리케이션과 파이프라인을 신속하게 구축할 수 있는 플랫폼
- 챗봇, 질의응답 시스템, 자동 요약 등 다양한 LLM 애플리케이션을 쉽게 개발할 수 있도록 지원하는 프레임워크
- 2024년 12월 LangChain 1.0을 정식 출시
- LLM(OpenAI, Anthropic, Gemini 등)을 중심으로
Prompt 관리 , LLM 호출, 문서 로딩, 임베딩, 벡터DB, 검색(Retriever), Agent, Memory, Tool, Output Parser 등을 하나의 프레임워크로 제공하는 라이브러리

langchain 1.0부터 주요 변화점 :

createAgent 라는 새로운 표준 에이전트 생성 방식 도입
미들웨어 기반 커스터마이징 강화
표준 콘텐츠 블록 지원 확대
패키지 구조 정리 (구형 기능은 @langchain/classic으로 이동)

LangChain 이란

■ LangChain

- langchain 1.0 - 5계층 아키텍처

계층	특성
1. AI 추론 및 생성 계층	다양한 LLM 모델(OpenAI, Anthropic, Google 등)을 표준화된 인터페이스로 사용 특정 공급자에 종속되지 않고 자유롭게 모델을 교체할 수 있습니다.
2. 외부 기능 계층	API, 데이터베이스, 웹 서비스 등 외부 시스템과 연동 Tool Calling을 통해 LLM이 정의된 함수를 직접 호출할 수 있습니다.
3. 오케스트레이션 계층	LangGraph를 기반으로 하는 에이전트(Agent)를 통해 복잡한 추론과 행동의 흐름을 제어 내구성 있는 실행(durable execution), 스트리밍, 영속성을 지원합니다.
4. 컨텍스트 보존 계층	메시지 히스토리, 커스텀 상태 스키마, 장기 메모리를 통해 대화의 맥락을 유지합니다. 토큰 한계를 넘지 않으면서 효과적으로 컨텍스트를 관리할 수 있습니다.
5. 정보 접근 계층	문서 로더, 텍스트 분할기, 벡터 스토어, 검색기(Retriever)를 통해 RAG(Retrieval-Augmented Generation) 패턴을 구현할 수 있습니다.

LangChain 이란

■ Middleware

- LLM 호출 전후 또는 Agent 실행 과정에서 공통 로직을 삽입하는 기능
- 모델 호출 전, 모델 호출 후, Tool 호출 전/후, Agent 실행 중 공통 기능을 수행하는 계층입니다.

미들웨어	설명
SummarizationMiddleware	대화가 길어지면 자동으로 요약하여 토큰 한계를 관리
HumanInTheLoopMiddleware	도구 실행 전 사람의 승인을 받는 기능
PIIMiddleware	개인정보(PII) 탐지 및 마스킹
ModelRetryMiddleware	시적 오류 발생 시 자동 재시도
Logging Middleware	질문, 응답, Tool 호출 로그 기록
Cost Middleware	토큰 사용량 및 비용 집계
Guardrail Middleware	유해한 질문 차단
Prompt Middleware	System Prompt 동적 추가
Authentication Middleware	사용자 인증 및 권한 검사
Cache Middleware	동일 질문 캐시 처리
Cache Middleware	호출 횟수 제한
Monitoring Middleware	LangSmith, OpenTelemetry 연동

LangChain 1.0 프레임워크 구성

■ langchain (메인 패키지)

- 에이전트 생성, 모델 초기화, 도구 정의 등 핵심 기능을 제공

```
from langchain.agents import create_agent          # 에이전트 생성
from langchain.chat_models import init_chat_model # 통합 모델 초기화
from langchain.tools import tool                  # 도구 데코레이터
```

■ langchain-core

- 메시지, 프롬프트, Runnable 등 기본 추상화와 인터페이스를 제공

```
from langchain_core.messages import SystemMessage, HumanMessage, AIMessage
from langchain_core.prompts import ChatPromptTemplate
```

LangChain 1.0 프레임워크 구성

■ langchain-community

- LLM 외부의 다양한 시스템(DB, Vector DB, 검색엔진, 문서로더, API 등)과 연결하는 수백 개의 커넥터를 모아놓은 라이브러리

분류	기능
Document Loader	PDF, Word, Excel, HTML, Markdown 등 문서 읽기
Vector Store	FAISS, Chroma, Milvus, Weaviate, Pinecone 등 연동
Retriever	BM25, Wikipedia, Arxiv 등 검색
Embedding	일부 커뮤니티 임베딩 모델 지원
Database	PostgreSQL, MySQL, SQLite 등
Search	Tavily, DuckDuckGo 등
Tool	Wikipedia, Shell, Python REPL 등
Callback	로그 및 추적 기능
Utility	SQL, 파일 처리 등

LangChain 1.0 프레임워크 구성

■ langserve

- LangChain으로 만든 Chain, Runnable, Agent를 REST API 형태로 배포할 수 있도록 지원하는 라이브러리
- LangChain을 웹 서비스(API)로 만들어주는 역할
- 브라우저에서 바로 테스트 가능한 Playground를 제공

```
from fastapi import FastAPI
from langserve import add_routes

from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

app = FastAPI()
llm = ChatOpenAI()
prompt = ChatPromptTemplate.from_template( "Translate {text} to Korean.")
chain = prompt | llm | StrOutputParser()
add_routes(
    app,
    chain,
    path="/translate"
)
```


LangChain 1.0 프레임워크 구성

■ 프로바이더 패키지 (langchain-openai, langchain-anthropic 등)

- 각 LLM 제공자와의 통합을 담당

```
from langchain_openai import ChatOpenAI
from langchain_anthropic import ChatAnthropic
from langchain_google_genai import ChatGoogleGenerativeAI
```

■ LangGraph

- 에이전트의 실행 런타임을 제공
- create_agent로 생성된 에이전트는 내부적으로 LangGraph 위에서 동작합니다.

■ LangSmith

- 에이전트의 실행을 추적, 디버깅, 평가하는 관측성(Observability) 플랫폼

Messages

■ Message

- 모델과의 상호작용에서 가장 기본이 되는 단위
- 모든 대화는 메시지의 시퀀스로 구성
- 각 메시지는 역할(role)과 내용(content)을 가집니다.

메시지의 구성 요소	설명
Role	메시지의 발신자 유형, system, user, assistant, tool
Content	실제 메시지 내용 텍스트, 이미지, 파일 등
Metadata	부가 정보, 토큰 수, 응답 ID 등

Messages

■ Messages

- langchain_core.messages 패키지

메시지	역할	설명
BaseMessage	모든 메시지의 부모(추상)	직접 사용하지 않음, 상속용
SystemMessage	시스템 프롬프트(지침, 역할 부여)	대화 맨 앞에서 LLM의 행동 방식 설정 대화의 맥락, 역할, 제약사항 등을 설정
HumanMessage	사용자의 입력	사람이 작성한 질문/요청 이미지 등 멀티모달 콘텐츠를 포함할 수 있습니다.
AIMessage	LLM(모델)의 응답	모델이 생성한 답변, tool_calls 포함 가능
ToolMessage	Tool 실행 결과	AI가 호출한 tool의 실행 결과를 다시 모델에 전달
FunctionMessage	함수 호출 결과	OpenAI function-calling 초기 버전 호환용, 현재는 ToolMessage로 대체 권장
ChatMessage		role을 자유롭게 지정 가능한 범용 메시지
AIMessageChunk ToolMessageChunk	스트리밍 응답 조각	.stream() 사용 시 토큰 단위로 전달되는 메시지

Messages

■ Messages

■ BaseMessage 공통 속성

속성	설명
content	메시지 본문 (텍스트 또는 멀티모달 콘텐츠 블록 리스트)
type	메시지 종류 식별자 ("human", "ai", "system", "tool" 등)
id	메시지 고유 식별자 (선택)
name	발신자 이름 (선택, 여러 사용자/에이전트 구분 시 사용)
additional_kwargs	모델별 부가 정보 저장 (예: function_call 등 비표준 필드)
response_metadata	응답 관련 메타데이터 (토큰 사용량, finish_reason 등, 주로 AIMessage에서 채워짐)

■ ChatMessage 전용 속성

속성	설명
role	자유롭게 지정 가능한 역할명 (예: "narrator", "reviewer" 등)

Messages

■ Messages

- AIMessage 전용 속성

속성	설명
tool_calls	모델이 호출하기로 결정한 도구 목록 (name, args, id 포함)
invalid_tool_calls	파싱 실패한 tool call 정보
usage_metadata	입력/출력 토큰 수 등 사용량 정보

- ToolMessage 전용 속성

속성	설명
tool_call_id	어떤 tool_call에 대한 응답인지 매칭하는 ID
content	tool 실행 결과 (보통 문자열, 경우에 따라 list/dict)
status	"success" 또는 "error" (실행 성공 여부)

- Message클래스 모두는 Pydantic BaseModel 기반이라 .model_dump(), .dict() 등으로 직렬화 가능합니다.

Messages

■ Messages

```
# 기본 시스템 메시지
from langchain_core.messages import SystemMessage

system_msg = SystemMessage(content="당신은 친절한 한국어 AI 어시스턴트입니다.")

# 상세한 역할 정의
system_msg = SystemMessage(content="""당신은 파이썬 프로그래밍 전문가입니다.
- 초보자도 이해할 수 있게 설명하세요
- 코드 예제를 포함하세요
- 한국어로 답변하세요""")
```

Messages

■ Messages

```
from langchain_core.messages import HumanMessage

# 텍스트 메시지
human_msg = HumanMessage(content="파이썬 리스트 컴프리헨션을 설명해주세요.")

# 이름 포함 (선택)
human_msg = HumanMessage(
    content="안녕하세요!",
    name="user_123"
)
```

Messages

■ Messages

```
#URL 기반 이미지 입력
from langchain.chat_models import init_chat_model
from langchain_core.messages import HumanMessage

model = init_chat_model("gpt-4o")

# 이미지 URL 포함
message = HumanMessage(
    content=[
        {"type": "text", "text": "이 이미지에 무엇이 있나요?"},
        {
            "type": "image_url",
            "image_url": {"url": "https://example.com/image.jpg"}
        },
    ]
)

response = model.invoke([message])
print(response.content)
```


Messages

■ Messages

```
#로컬 이미지를 Base64로 인코딩하여 전송
import base64
from langchain.chat_models import init_chat_model
from langchain_core.messages import HumanMessage
# 이미지 파일을 Base64로 인코딩
def encode_image(image_path: str) -> str:
    with open(image_path, "rb") as image_file:
        return base64.b64encode(image_file.read()).decode("utf-8")

image_data = encode_image("./my_image.png")
model = init_chat_model("gpt-4o")
message = HumanMessage(
    content=[
        {"type": "text", "text": "이 다이어그램을 분석해주세요."},
        {
            "type": "image_url",
            "image_url": {
                "url": f"data:image/png;base64,{image_data}"
            }
        }
    ],
)
response = model.invoke([message])
print(response.content)
```

Messages

■ Messages

```
#여러 이미지 동시 처리
message = HumanMessage(
    content=[
        {"type": "text", "text": "두 이미지를 비교해주세요."},
        {
            "type": "image_url",
            "image_url": {"url": "https://example.com/image1.jpg"}
        },
        {
            "type": "image_url",
            "image_url": {"url": "https://example.com/image2.jpg"}
        },
    ]
)
```

Messages

■ Messages

```
#이미지 처리 상세도를 설정
message = HumanMessage(
    content=[
        {"type": "text", "text": "이 문서를 읽어주세요."},
        {
            "type": "image_url",
            "image_url": {
                "url": "https://example.com/document.png",
                "detail": "high" # low, high, auto
            }
        },
    ],
)
```

Messages

■ Messages

```
from langchain.chat_models import init_chat_model
from langchain_core.messages import HumanMessage

model = init_chat_model("gpt-4o-mini")

# 모델 호출 - AIMessage 반환
response = model.invoke([HumanMessage(content="안녕하세요!")])

print(type(response)) # AIMessage
print(f"내용: {response.content}")
print(f"토큰 사용량: {response.usage_metadata}")
print(f"응답 메타데이터: {response.response_metadata}")
```

Messages

■ Messages

- AIMessage는 도구 호출 정보를 포함할 수 있습니다

```
if response.tool_calls:  
    for tool_call in response.tool_calls:  
        print(f"도구 이름: {tool_call['name']}")  
        print(f"도구 인자: {tool_call['args']}")
```

```
from langchain_core.messages import ToolMessage  
  
tool_msg = ToolMessage(  
    content="서울의 현재 온도는 25도입니다.",  
    tool_call_id="call_abc123",  
    name="get_weather"  
)
```

Messages

■ Messages

- 모델에 메시지 리스트를 전달하여 대화 컨텍스트를 구성

```
from langchain.chat_models import init_chat_model
from langchain_core.messages import SystemMessage, HumanMessage, AIMessage

model = init_chat_model("gpt-4o-mini")

# 대화 히스토리 구성
messages = [
    SystemMessage(content="당신은 요리 전문가입니다."),
    HumanMessage(content="김치찌개 만드는 법을 알려주세요."),
    AIMessage(content="김치찌개는 다음과 같이 만듭니다..."),
    HumanMessage(content="고기 없이 만들 수 있나요?"),
]

# 컨텍스트를 유지한 응답
response = model.invoke(messages)
print(response.content)
```

Messages

■ Messages

- 메시지 객체 대신 딕셔너리 형식을 사용하여 대화 컨텍스트를 구성

```
from langchain.chat_models import init_chat_model

model = init_chat_model("gpt-4o-mini")

# 딕셔너리 형식의 메시지
messages = [
    {"role": "system", "content": "당신은 유용한 AI 어시스턴트입니다."},
    {"role": "user", "content": "파이썬이란?"},
    {"role": "assistant", "content": "파이썬은 프로그래밍 언어입니다."},
    {"role": "user", "content": "주요 특징은?"},
]

response = model.invoke(messages)
print(response.content)
```

Prompt

Prompt

■ Prompt 작성 원칙

클래스	설명
PromptTemplate	기본적이고 범용적인 프롬프트, 문자열 + {변수} 형식
SystemMessagePromptTemplate	System Message용, System 역할 부여
HumanMessagePromptTemplate	Human(User) Message용, 사용자 메시지
AIMessagePromptTemplate	AI Assistant Message용, AI 이전 응답
ChatPromptTemplate	Chat 모델 전용 (메시지 리스트), 여러 메시지 조합
FewShotPromptTemplate	Few-shot 학습용, 예시 포함
FewShotChatMessagePromptTemplate	Chat Few-shot용, Chat + Few-shot
PipelinePromptTemplate	여러 프롬프트를 조합
StringPromptTemplate	커스텀 프롬프트 베이스 클래스

Prompt

■ Prompt 작성 원칙

- LLM(대규모 언어 모델)에 전달할 입력
- 변수 치환, 대화 기록 관리, 예제(Few-shot) 삽입, 메시지 구성 등을 지원

원칙	설명
명확성	모호함 제거, 구체적 지시, 상세한 템플릿 작성 예시] "설명해줘" → "3가지 핵심 포인트로 설명해줘"
구체성	세부 사항과 제약 조건 명시 예시] "요약해줘" → "200자 이내로 요약해줘"
컨텍스트	필요한 배경 정보 제공, System 메시지 활용 예시] 대상 청중, 사용 목적 명시
구조화	역할, 지시, 예시 분리, ChatPromptTemplate 예시] System/Human 메시지 분리
출력 형식	원하는 응답 형식 지정 예시] JSON, 표, 번호 목록 등
예시 제공	Few-shot으로 패턴 학습, FewShotChatMessagePromptTemplate 예시] 입력-출력 예시 포함
환각 방지	불확실성 인정 유도, 제약조건 명시

Prompt

■ PromptTemplate

- 단일 문장 또는 간단한 명령을 입력하여 단일 문장 또는 간단한 응답을 생성하는 데 사용되는 프롬프트
- 변수를 포함한 문자열 템플릿으로, 동적으로 프롬프트를 생성할 수 있게 함
- `from_template()`를 사용하여 문자열 템플릿으로부터 PromptTemplate 인스턴스를 생성
- `format()`를 사용하여, 변수에 값으로 템플릿에 채워서 프롬프트를 구성

```
from langchain_core.prompts import PromptTemplate

# 템플릿 정의
template = PromptTemplate.from_template(
    "{topic}에 대해 {length}자 이내로 설명해주세요."
)

# 변수 대입
prompt = template.format(topic="인공지능", length="100")
print(prompt)
# 출력: 인공지능에 대해 100자 이내로 설명해주세요.
```

Prompt

■ PromptTemplate

- "지시", "예시", "맥락", "질문" 과 같은 다양한 구성요소들을 조합

구분	내용 & 예시
지시	언어 모델에게 어떤 작업을 수행하도록 요청하는 구체적인 지시 "아래 제공된 제품 리뷰를 요약해주세요."
예시	요청된 작업을 수행하는 방법에 대한 하나 이상의 예시 "예를 들어, '이 제품은 매우 사용하기 편리하며 배터리 수명이 길다'라는 리뷰는 '사용 편리성과 긴 배터리 수명이 특징'으로 요약할 수 있습니다."
맥락	특정 작업을 수행하기 위한 추가적인 맥락 "리뷰는 스마트워치에 대한 것이며, 사용자 경험에 초점을 맞추고 있습니다."
질문	어떤 답변을 요구하는 구체적인 질문 "이 리뷰를 바탕으로 스마트워치의 주요 장점을 두세 문장으로 요약해주세요."

Prompt

■ PromptTemplate

- + 연산자를 사용하여 직접 결합하는 동작을 지원
- PromptTemplate 인스턴스 간의 직접적인 결합, 인스턴스와 문자열로 이루어진 템플릿을 결합하여 새로운 PromptTemplate 인스턴스를 생성하는 것도 가능

```
# 문자열 템플릿 결합 (PromptTemplate + PromptTemplate + 문자열)
combined_prompt = (
    prompt_template
    + PromptTemplate.from_template("\n\n아버지를 아버지라 부를 수 없습니다.")
    + "\n\n{language}로 번역해주세요."
)

combined_prompt
```

Prompt

■ ChatPromptTemplate

- 대화형 상황에서 여러 메시지 입력을 기반으로 단일 메시지 응답을 생성하는 데 사용
- 챗봇 개발에 주로 사용

```
##명확성과 구체성
from langchain.chat_models import init_chat_model
from langchain_core.prompts import ChatPromptTemplate

llm = init_chat_model("gpt-4o-mini")

specific_prompt = ChatPromptTemplate.from_template(
    """"{topic}에 대해 다음 형식으로 설명해주세요:

1. 정의 (1-2문장)
2. 핵심 특징 3가지
3. 실제 활용 사례 2가지

전문 용어는 피하고 초보자도 이해할 수 있게 작성해주세요.""")
chain = specific_prompt | llm
result = chain.invoke({"topic": "머신러닝"})
```

Prompt

■ ChatPromptTemplate

- `from_messages()`를 사용하여 메시지 리스트로부터 `ChatPromptTemplate`인스턴스를 생성
- 튜플 형태의 메시지 리스트를 입력 받아, 각 메시지의 역할(`type`)과 내용(`content`)을 기반으로 프롬프트를 구성합니다
- `format_messages()`는 사용자의 입력을 프롬프트에 동적으로 삽입하여, 최종적으로 대화형 상황을 반영한 메시지 리스트를 생성

```
# 튜플 형태의 메시지 목록으로 프롬프트 생성 (type, content)

from langchain_core.prompts import ChatPromptTemplate

chat_prompt = ChatPromptTemplate.from_messages([
    ("system", "이 시스템은 천문학 질문에 답변할 수 있습니다."),
    ("user", "{user_input}"),
])

messages = chat_prompt.format_messages(user_input="태양계에서 가장 큰 행성은 무엇인가요?")
messages
```

Prompt

■ ChatPromptTemplate

- SystemMessagePromptTemplate와 HumanMessagePromptTemplate를 사용하여 대화형 프롬프트를 생성

```
# MessagePromptTemplate 활용

from langchain_core.prompts import SystemMessagePromptTemplate, HumanMessagePromptTemplate

chat_prompt = ChatPromptTemplate.from_messages(
    [
        SystemMessagePromptTemplate.from_template("이 시스템은 천문학 질문에 답변할 수 있습니다."),
        HumanMessagePromptTemplate.from_template("{user_input}"),
    ]
)

messages = chat_prompt.format_messages(user_input="태양계에서 가장 큰 행성은 무엇인가요?")
messages
```


Prompt

■ PromptTemplate

■ 제약 조건 명시

```
from langchain_core.prompts import ChatPromptTemplate

# 출력 제약 조건을 명확히 지정
prompt = ChatPromptTemplate.from_messages([
    ("system", """"당신은 기술 문서 작성 전문가입니다.

응답 규칙:
- 문장은 간결하게 (20단어 이내)
- 능동태 사용
- 전문 용어 사용 시 괄호 안에 설명 추가
- 총 200자 이내로 작성"""),
    ("human", "{question}")
])
```

Prompt

■ PromptTemplate

- 컨텍스트 제공 - 배경 정보 포함

```
from langchain_core.prompts import ChatPromptTemplate

# 컨텍스트 없는 프롬프트
without_context = "파이썬 웹 프레임워크를 추천해줘."

# 컨텍스트 포함 프롬프트
with_context = ChatPromptTemplate.from_messages([
    ("system", "당신은 시니어 백엔드 개발자입니다."),
    ("human", """"다음 프로젝트 요구사항에 맞는 파이썬 웹 프레임워크를 추천해주세요.

프로젝트 정보:
- 팀 규모: 3명 (주니어 2, 시니어 1)
- 예상 트래픽: 일 10만 요청
- 주요 기능: REST API, 실시간 알림
- 기한: 3개월

각 프레임워크의 장단점과 이 프로젝트에 적합한 이유를 설명해주세요.""")
])
```

Prompt

■ PromptTemplate

- 컨텍스트 제공 - 대상 청중 명시

```
from langchain_core.prompts import ChatPromptTemplate

prompt = ChatPromptTemplate.from_messages([
    ("system", """"당신은 교육 콘텐츠 전문가입니다.

대상 청중: {audience}
- 초보자: 비유와 일상 예시 사용, 전문 용어 최소화
- 중급자: 개념과 실습 코드 병행
- 전문가: 심화 내용과 최적화 기법 포함"""),
    ("human", "{topic}에 대해 설명해주세요.")
])

# 대상에 따라 다른 응답
chain = prompt | llm
beginner = chain.invoke({"audience": "초보자", "topic": "API"})
expert = chain.invoke({"audience": "전문가", "topic": "API"})
```

Prompt

■ PromptTemplate

- 구조화된 프롬프트 - 역할(Role) 설정

```
from langchain_core.prompts import ChatPromptTemplate

# System 메시지로 역할 정의
prompt = ChatPromptTemplate.from_messages([
    ("system", """당신은 10년 경력의 데이터 사이언티스트입니다.

전문 분야:
- 머신러닝 모델 개발
- 데이터 분석 및 시각화
- Python, SQL, TensorFlow

응답 스타일:
- 데이터 기반 설명
- 코드 예시 포함
- 실무 관점의 조언"""),
    ("human", "{question}")
])
```

Prompt

■ PromptTemplate

- 구조화된 프롬프트 - 단계별 지시사항

```
from langchain_core.prompts import ChatPromptTemplate
```

```
prompt = ChatPromptTemplate.from_messages([  
    ("system", "당신은 코드 리뷰 전문가입니다."),  
    ("human", """"다음 코드를 리뷰해주세요:
```

```
<code>  
{code}  
</code>
```

다음 순서로 분석해주세요:

1. ****기능 요약****: 코드가 수행하는 작업 설명
2. ****장점****: 잘 작성된 부분 (2-3가지)
3. ****개선점****: 수정이 필요한 부분 (2-3가지)
4. ****리팩토링 제안****: 개선된 코드 예시

```
각 섹션은 명확히 구분하여 작성해주세요.""")  
])
```

Prompt

■ Few-shot Template

- Few-shot 학습은 언어 모델에 몇 가지 예시를 제공하여 특정 작업을 수행하도록 유도하는 기법
- Few-shot 학습을 활용함으로써, 언어 모델은 주어진 예제들을 참고하여 더 정확하고 일관된 응답을 생성할 수 있습니다.
- 특히 특정 도메인이나 형식의 질문에 대해 모델의 성능을 향상시키는 데 효과적
- FewShotPromptTemplate 은 예제들을 결합하고 새로운 입력을 추가하여 최종 프롬프트를 생성합니다.

```
from langchain_core.prompts import PromptTemplate

example_prompt = PromptTemplate.from_template("질문: {question}\n{answer}")
examples = [
    {
        "question": "지구의 대기 중 가장 많은 비율을 차지하는 기체는 무엇인가요?",
        "answer": "지구 대기의 약 78%를 차지하는 질소입니다."
    },
    {
        "question": "광합성에 필요한 주요 요소들은 무엇인가요?",
        "answer": "광합성에 필요한 주요 요소는 빛, 이산화탄소, 물입니다."
    },
    .....
]
```

Prompt

■ Few-shot Template

```
from langchain_core.prompts import FewShotPromptTemplate

# FewShotPromptTemplate을 생성합니다.
prompt = FewShotPromptTemplate(
    examples=examples,          # 사용할 예제들
    example_prompt=example_prompt, # 예제 포매팅에 사용할 템플릿
    suffix="질문: {input}",      # 예제 뒤에 추가될 접미사
    input_variables=["input"],   # 입력 변수 지정
)

# 새로운 질문에 대한 프롬프트를 생성하고 출력합니다.
print(prompt.invoke({"input": "화성의 표면이 붉은 이유는 무엇인가요?"}).to_string())
```

Prompt

■ SemanticSimilarityExampleSelector

- 더 관련성 높은 컨텍스트를 제공하여, 입력 질문과 가장 유사한 예제를 선택하여 모델의 응답 품질을 향상시킬 수 있습니다.

```
from langchain_chroma import Chroma
from langchain_core.example_selectors import SemanticSimilarityExampleSelector
from langchain_openai import OpenAIEmbeddings

example_selector = SemanticSimilarityExampleSelector.from_examples(
    examples,          # 사용할 예제들
    OpenAIEmbeddings(), # 임베딩 모델
    Chroma,            # 벡터 저장소
    k=1,               # 선택할 예제 수
)

# 새로운 질문에 대해 가장 유사한 예제를 선택합니다.
question = "화성의 표면이 붉은 이유는 무엇인가요?"
selected_examples = example_selector.select_examples({"question": question})
print(f"입력과 가장 유사한 예제: {question}")
for example in selected_examples:
    print("\n")
    for k, v in example.items():
        print(f"{k}: {v}")
```


Prompt

■ PromptTemplate

■ 출력 형식 지정

```
# 표 형식 요청
table_prompt = ChatPromptTemplate.from_messages([
    ("system", "응답은 마크다운 표 형식으로 작성하세요."),
    ("human", """"다음 프로그래밍 언어들을 비교해주세요: {languages}
| 언어 | 주요 용도 | 장점 | 단점 | 학습 난이도 |
형식으로 작성해주세요.""")
])

# JSON 형식 요청
json_prompt = ChatPromptTemplate.from_messages([
    ("system", "응답은 유효한 JSON 형식으로만 작성하세요. 다른 텍스트는 포함하지 마세요."),
    ("human", """"다음 텍스트에서 정보를 추출하세요: {text}
형식:
{{
  "name": "이름",
  "email": "이메일",
  "phone": "전화번호"
}}""")
])

# 번호 목록 형식 요청
list_prompt = ChatPromptTemplate.from_messages([
    ("system", "응답은 번호 목록으로 작성하세요. 각 항목은 한 줄로 간결하게."),
    ("human", "{topic}의 장점 5가지를 알려주세요.")
])
```

Prompt

■ PromptTemplate

- 구조화된 출력과 결합

```
from langchain.chat_models import init_chat_model
from pydantic import BaseModel, Field

class CodeReview(BaseModel):
    """코드 리뷰 결과"""
    summary: str = Field(description="코드 기능 요약")
    score: int = Field(description="1-10점 품질 점수")
    issues: list[str] = Field(description="발견된 문제점 목록")
    suggestions: list[str] = Field(description="개선 제안 목록")

llm = init_chat_model("gpt-4o-mini")
structured_llm = llm.with_structured_output(CodeReview)

result = structured_llm.invoke("""
다음 코드를 리뷰해주세요:

def calc(x):
    return x*2+1
""")

print(f"점수: {result.score}/10")
print(f"문제점: {result.issues}")
```

Prompt

■ PromptTemplate

- 환각 방지

```
from langchain_core.prompts import ChatPromptTemplate

prompt = ChatPromptTemplate.from_messages([
    ("system", """"당신은 정확한 정보만 제공하는 AI입니다.

중요한 규칙:
1. 확실하지 않은 정보는 "확실하지 않습니다"라고 말하세요
2. 추측이 필요한 경우 "추측입니다만..."으로 시작하세요
3. 최신 정보가 필요한 질문은 검증이 필요함을 안내하세요
4. 출처가 있다면 명시하세요"""),
    ("human", "{question}")
])
```

Prompt

■ Partial Prompt

- 프롬프트 템플릿 부분 포매팅은 필요한 값의 일부만 미리 입력하여 새로운 프롬프트 템플릿을 만드는 방법입니다.

```
#문자열을 사용한 부분 포매팅
from langchain_core.prompts import PromptTemplate

# 기본 프롬프트 템플릿 정의
prompt = PromptTemplate.from_template("지구의 {layer}에서 가장 흔한 원소는 {element}입니다.")

# 'layer' 변수에 '지각' 값을 미리 지정하여 부분 포매팅
partial_prompt = prompt.partial(layer="지각")

# 나머지 'element' 변수만 입력하여 완전한 문장 생성
print(partial_prompt.format(element="산소"))
```

Prompt

■ Partial Prompt

```
# 프롬프트 초기화 시 부분 변수 지정
prompt = PromptTemplate(
    template="지구의 {layer}에서 가장 흔한 원소는 {element}입니다.",
    input_variables=["element"], # 사용자 입력이 필요한 변수
    partial_variables={"layer": "맨틀"} # 미리 지정된 부분 변수
)

# 남은 'element' 변수만 입력하여 문장 생성
print(prompt.format(element="규소"))
```

Prompt

■ Partial Prompt

- 함수를 사용한 부분 포매팅 - 동적으로 값을 생성.

```
# 현재 계절을 반환하는 함수 정의
def get_current_season():
    month = datetime.now().month
    if 3 <= month <= 5:
        return "봄"
    elif 6 <= month <= 8:
        return "여름"
    elif 9 <= month <= 11:
        return "가을"
    else:
        return "겨울"

# 함수를 사용한 부분 변수가 있는 프롬프트 템플릿 정의
prompt = PromptTemplate(
    template="{season}에 일어나는 대표적인 지구과학 현상은 {phenomenon}입니다.",
    input_variables=["phenomenon"], # 사용자 입력이 필요한 변수
    partial_variables={"season": get_current_season} # 함수를 통해 동적으로 값을 생성하는 부분 변수
)

# 'phenomenon' 변수만 입력하여 현재 계절에 맞는 문장 생성
print(prompt.format(phenomenon="꽃가루 증가"))
```

Langchain Model

LangChain 모델 유형

■ LLM

- LLM 클래스는 사용자가 특정 주제에 대한 설명을 요청할 때, LLM은 주어진 텍스트 입력을 바탕으로 상세한 설명을 생성하여 반환
- LLM은 주로 단일 요청에 대한 복잡한 출력을 생성
- LLM은 광범위한 언어 이해 및 텍스트 생성 작업에 사용
- LLM은 문서 요약, 콘텐츠 생성, 질문에 대한 답변 생성 등 복잡한 자연어 처리 작업을 수행
- Chat Model은 사용자와의 상호작용을 통한 연속적인 대화를 관리

■ Chat Model

- Chat Model 클래스는 메시지의 리스트를 입력으로 받고, 하나의 메시지를 반환
- 대화형 상황에 최적화되어 있으며, 사용자와의 연속적인 대화를 처리하는 데 사용
- 대화의 맥락을 유지하면서 적절한 응답을 생성하는 데 중점을 둡니다.
- 사용자가 챗봇과 대화하는 상황에서, 사용자의 질문과 이전 대화 내용을 고려하여 적절한 답변을 생성합니다.

LangChain 모델 유형

■ LLM 표준 인터페이스

- OpenAI, Cohere, Hugging Face 등과 같은 다양한 LLM 제공 업체로부터 모델을 사용할 수 있게 하는 플랫폼 역할을 한다
- 랭체인(LangChain)의 LLM 클래스는 사용자가 문자열을 입력으로 제공하면, 그에 대한 응답으로 문자열을 반환하는 표준화된 방식을 제공
- 다양한 LLM 제공 업체 간의 호환성을 보장하며, 사용자는 복잡한 API 변환 작업 없이 여러 LLM을 쉽게 탐색하고 사용
- 랭체인은 OpenAI의 GPT 시리즈, Cohere의 LLM, Hugging Face의 Transformer 모델 등 다양한 LLM 제공 업체와의 통합을 지원

LangChain은 동기(sync), 비동기(async), 배치(batching), 스트리밍(streaming) 모드에서 모델을 사용할 수 있는 기능을 제공합니다.

LangChain은 다양한 모델 프로바이더(OpenAI, Anthropic, Google 등)를 지원

LangChain 모델 유형

■ ChatOpenAI

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI

chat = ChatOpenAI()

chat_prompt = ChatPromptTemplate.from_messages([
    ("system", "이 시스템은 여행 전문가입니다."),
    ("user", "{user_input}"),
])

chain = chat_prompt | chat
chain.invoke({"user_input": "안녕하세요? 한국의 대표적인 관광지 3군데를 추천해주세요."})
```

LangChain 모델 유형

■ 통합 모델 초기화 (init_chat_model)

- LangChain은 각 프로바이더마다 별도의 클래스(ChatOpenAI, ChatAnthropic 등)를 사용할 수 있지만, init_chat_model 함수를 사용하면 통합된 인터페이스로 모든 프로바이더의 모델을 초기화할 수 있습니다.

```
from langchain.chat_models import init_chat_model

# OpenAI (기본적으로 gpt- 접두사는 OpenAI로 인식)
openai_model = init_chat_model("gpt-4.1")

# Anthropic Claude
claude_model = init_chat_model("claude-sonnet-4-5-20250929")

# Google Gemini (프로바이더:모델명 형식)
gemini_model = init_chat_model("google_genai:gemini-2.5-flash-lite")

# AWS Bedrock (model_provider 파라미터 사용)
bedrock_model = init_chat_model(
    "anthropic.claude-3-5-sonnet-20240620-v1:0",
    model_provider="bedrock_converse"
)
```

LangChain 모델 유형

■ 통합 모델 초기화 (init_chat_model)

프로바이더	패키지	환경변수	예시
OpenAI	langchain-openai	OPENAI_API_KEY	gpt-4.1, gpt-4.1-mini
Anthropic	langchain-anthropic	ANTHROPIC_API_KEY	claude-sonnet-4-5-20250929
Google	langchain-google-genai	GOOGLE_API_KEY	google_genai:gemini-2.5-flash-lite
AWS Bedrock	langchain-aws	AWS 자격증명	bedrock_converse:anthropic.claude-3-5-sonnet
Azure OpenAI	langchain-openai	AZURE_OPENAI_API_KEY	azure_openai:gpt-

LangChain 모델 유형

■ init_chat_model 의 주요 파라미터

파라미터	패키지
model	None(기본값:None)
model_provider	None(기본값:None)
configurable_fields	런타임에 설정 가능한 필드를 지정. • None: 고정 모델 (기본) • 'any': 모든 필드 (보안 주의) • ["model", "temperature"]: 특정 필드만 모델을 동적으로 바꾸고 싶을 때 매우 유용
config_prefix	None(기본값:None)
**kwargs	해당 모델 클래스의 __init__에 전달되는 인자들. 자주 사용하는 옵션: • temperature: 생성의 무작위성 (0~2) • max_tokens: 최대 출력 토큰 수 • timeout: 타임아웃 (초) • max_retries: 재시도 횟수 • base_url: 커스텀 엔드포인트 • api_key: API 키 (환경변수로 설정하는 게 일반적)

LangChain 모델 유형

■ Model 실행 메서드

메서드	설명
invoke()	단일 요청
ainvoke()	FastAPI, asyncio 환경
batch()	여러 요청 한 번에 처리
abatch()	비동기
stream()	토큰 스트리밍
astream()	비동기
astream_events()	비동기 , 이벤트 스트림반환, 디버깅 모니터링

LangChain 모델 유형

■ 통합 모델 초기화 (init_chat_model)

- temperature, max_tokens 등의 파라미터를 설정
- 런타임에 모델을 동적으로 변경할 수 있다

```
from langchain.chat_models import init_chat_model
# 설정 가능한 모델 생성 (모델명을 지정하지 않으면 런타임에 지정 가능)
configurable_model = init_chat_model(temperature=0)
# GPT-4.1으로 실행
response_gpt = configurable_model.invoke(
    "당신의 이름은 무엇인가요?",
    config={"configurable": {"model": "gpt-4.1-nano"}}
)
# Claude로 실행 (같은 모델 인스턴스 사용)
response_claude = configurable_model.invoke(
    "당신의 이름은 무엇인가요?",
    config={"configurable": {"model": "claude-sonnet-4-5-20250929"}}
)
print(f"GPT: {response_gpt.content}")
print(f"Claude: {response_claude.content}")
```

LangChain 모델 유형

■ 통합 모델 초기화 (init_chat_model)

- 여러 모델을 체인에서 사용할 때, configurable_fields와 config_prefix를 활용하면 각 모델을 개별적으로 설정할 수 있습니다.

```
from langchain.chat_models import init_chat_model
# 첫 번째 모델 (요약용)
summary_model = init_chat_model(
    model="gpt-4.1-mini",
    temperature=0,
    configurable_fields=("model", "model_provider", "temperature", "max_tokens"),
    config_prefix="summary"
)
# 두 번째 모델 (창작용)
creative_model = init_chat_model(
    model="gpt-4.1",
    temperature=0.9,
    configurable_fields=("model", "model_provider", "temperature"),
    config_prefix="creative"
)
```


LangChain 모델 유형

■ 통합 모델 초기화 (init_chat_model)

- 여러 모델을 체인에서 사용할 때, configurable_fields와 config_prefix를 활용하면 각 모델을 개별적으로 설정할 수 있습니다.

```
# 런타임에 각 모델의 설정을 개별 변경
response = summary_model.invoke(
    "이 텍스트를 요약해주세요: ...",
    config={
        "configurable": {
            "summary_model": "claude-sonnet-4-5-20250929",
            "summary_temperature": 0.2,
            "summary_max_tokens": 500
        }
    }
)
```

LangChain 모델 유형

■ 통합 모델 초기화 (init_chat_model)

- 환경에 따라 다른 모델을 사용하는 패턴.

```
import os
from langchain.chat_models import init_chat_model

# 환경변수로 모델 설정
MODEL_NAME = os.getenv("LLM_MODEL", "gpt-4.1-mini")

model = init_chat_model(MODEL_NAME, temperature=0)

# 동일한 코드로 환경에 따라 다른 모델 사용
response = model.invoke("프로젝트 일정을 정리해주세요.")
```

```
# 개발 환경
export LLM_MODEL="gpt-4.1-mini"

# 프로덕션 환경
export LLM_MODEL="gpt-4.1"
```

LangChain 모델 유형

■ LLM 모델 파라미터

파라미터	패키지
temperature	생성된 텍스트의 다양성을 조정 값이 작으면 예측 가능하고 일관된 출력을 생성 값이 크면 다양하고 예측하기 어려운 출력을 생성
max_tokens	생성할 최대 토큰 수를 지정합니다. 생성할 텍스트의 길이를 제한합니다.
top_p (Top Probability)	생성 과정에서 특정 확률 분포 내에서 상위 P% 토큰만을 고려하는 방식 출력의 다양성을 조정
frequency_penalty	값이 클수록 이미 등장한 단어나 구절이 다시 등장할 확률을 감소시킵니다. 반복을 줄이고 텍스트의 다양성을 증가시킬 수 있습니다. (0~1)
presence_penalty	텍스트 내에서 단어의 존재 유무에 따라 그 단어의 선택 확률을 조정합니다. 값이 클수록 아직 텍스트에 등장하지 않은 새로운 단어를 사용 (0~1)
stop	특정 단어나 구절이 등장할 경우 생성을 멈추도록 설정 출력을 특정 포인트에서 종료하고자 할 때 사용됩니다.
n	생성할 응답 개수

LangChain 모델 유형

■ LLM 모델 파라미터

```
from langchain_openai import ChatOpenAI
# 모델 파라미터 설정
params = {
    "temperature" : 0.7,      # 생성된 텍스트의 다양성 조정
    "max_tokens" : 100,      # 생성할 최대 토큰 수
}
kwargs = {
    "frequency_penalty" : 0.5, # 이미 등장한 단어의 재등장 확률
    "presence_penalty" : 0.5,  # 새로운 단어의 도입을 장려
    "stop" : ["\n"]           # 정지 시퀀스 설정
}
# 모델 인스턴스를 생성할 때 설정
model = ChatOpenAI(model="gpt-4o-mini", **params, model_kwargs = kwargs)
# 모델 호출
question = "태양계에서 가장 큰 행성은 무엇인가요?"
response = model.invoke(input=question)
# 전체 응답 출력
print(response)
```

LangChain 모델 유형

■ LLM 모델 파라미터

- `bind()` : 사용하여 모델 인스턴스에 파라미터를 추가로 제공
특정 모델 설정을 기본값으로 사용하고자 할 때 유용하며, 특수한 상황에서 일부 파라미터를 다르게 적용하고 싶을 때 사용

```
from langchain_core.prompts import ChatPromptTemplate

prompt = ChatPromptTemplate.from_messages([
    ("system", "이 시스템은 천문학 질문에 답변할 수 있습니다."),
    ("user", "{user_input}"),
])

model = ChatOpenAI(model="gpt-4o-mini", max_tokens=100)
messages = prompt.format_messages(user_input="태양계에서 가장 큰 행성은 무엇인가요?")
before_answer = model.invoke(messages)
# # binding 이전 출력
print(before_answer)
# 모델 호출 시 추가적인 인수를 전달하기 위해 bind 메서드 사용 (응답의 최대 길이를 10 토큰으로 제한)
chain = prompt | model.bind(max_tokens=10)
after_answer = chain.invoke({"user_input": "태양계에서 가장 큰 행성은 무엇인가요?"})
# binding 이후 출력
print(after_answer)
```

LangChain 모델 유형

■ Anthropic의 Claude 모델

- Anthropic API 키를 `ANTHROPIC_API_KEY` 환경 변수로 설정하거나 `api_key` 매개변수로 직접 전달

```
pip install -U langchain-anthropic
```

```
# temperature=0으로 설정하여 결정론적인 출력을 얻고, max_tokens=200으로 응답 길이를 제한
from langchain_anthropic import ChatAnthropic
```

```
# 모델 로드
```

```
llm = ChatAnthropic(
    model="claude-haiku-4-5-20251001",
    temperature=0,
    max_tokens=200,
    api_key="인증키 입력"
)
ai_msg = llm.invoke(messages) print(ai_msg)
print(ai_msg.content)
```

LangChain 모델 유형

■ Gemini 모델

- Google AI Studio API 키를 `GOOGLE_API_KEY` 환경 변수로 설정하거나 `google_api_key` 매개변수로 직접 전달할 수 있습니다.

```
pip install -U langchain-google-genai
```

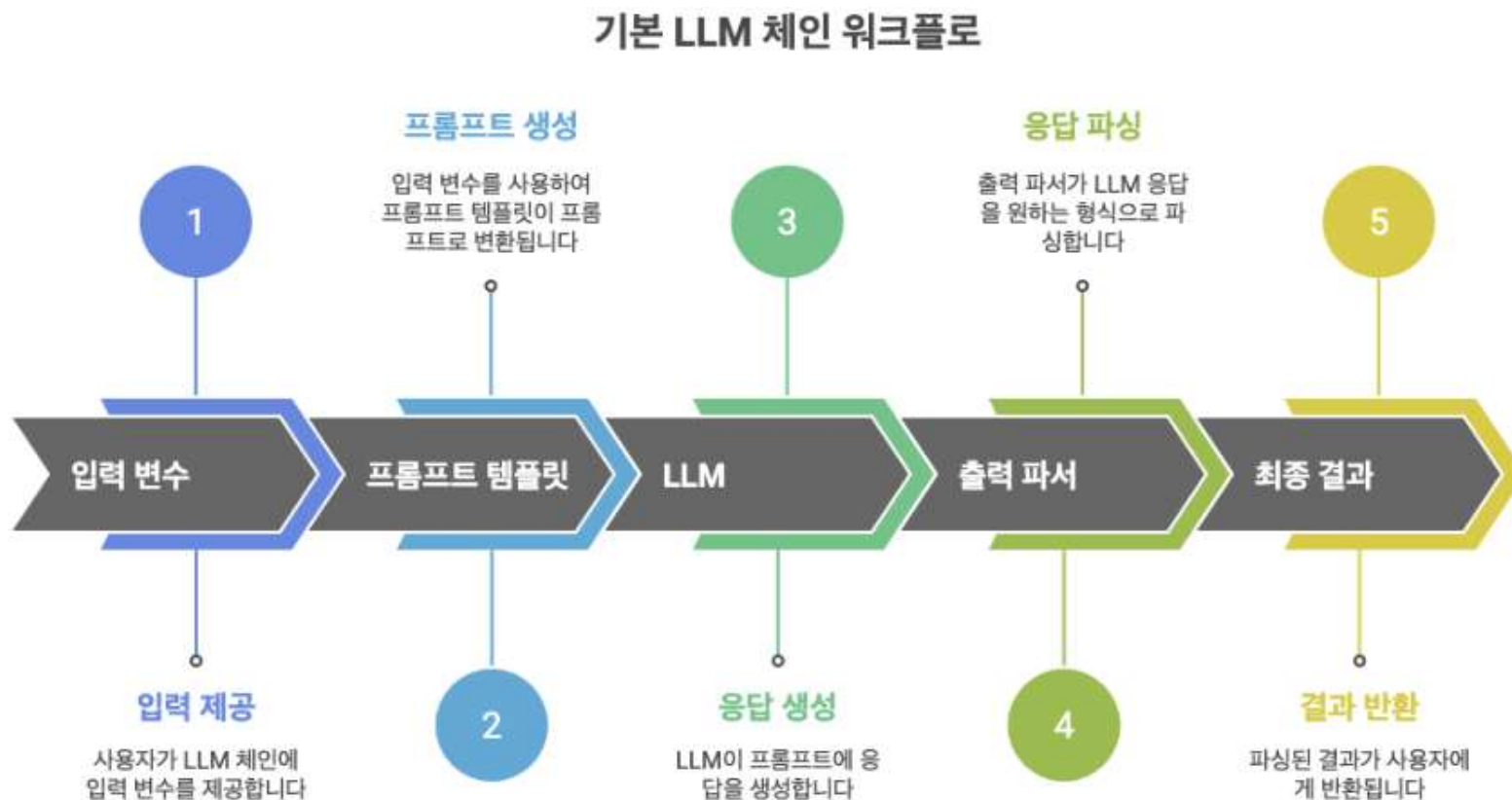
```
# temperature=0으로 설정하여 결정론적인 출력을 얻고, max_tokens=200으로 응답 길이를 제한
from langchain_google_genai import ChatGoogleGenerativeAI
# 모델 로드
llm = ChatGoogleGenerativeAI(
    model="gemini-2.5-flash",
    temperature=0,
    max_output_tokens=200,
    google_api_key="인증키 입력"
)
ai_msg = llm.invoke(messages)
print(ai_msg.content)
```

LLMChain

Chain

■ Chain

- 여러 컴포넌트를 연결하여 하나의 워크플로우를 구성하는 것



Chain

■ LCEL (LangChain Expression Language)

- LangChain 1.0에서 기존의 [LLMChain](#) 클래스는 langchain-classic 패키지로 이동

```
from langchain.chat_models import init_chat_model
from langchain_core.prompts import ChatPromptTemplate

# 프롬프트 템플릿
prompt = ChatPromptTemplate.from_template(
    "{topic}에 대해 간단히 설명해주세요."
)

# LLM
llm = init_chat_model("gpt-4o-mini")

# 체인 구성 (프롬프트 | LLM)
chain = prompt | llm

# 체인 실행
response = chain.invoke({"topic": "인공지능"})
print(response.content)
```

Chain

■ LCEL (LangChain Expression Language)

```
from langchain_core.output_parsers import StrOutputParser

# 체인에 출력 파서 추가
chain = prompt | llm | StrOutputParser()

# 문자열로 바로 결과 반환
result = chain.invoke({"topic": "머신러닝"})
print(result) # 문자열
```

Chain

■ Chain 실행 방식

메서드	설명	사용 사례
invoke()	단일 입력 동기 실행	일반적인 호출
batch()	여러 입력 병렬 실행	대량 처리
stream()	토큰 단위 스트리밍	실시간 UI
ainvoke()	비동기 실행	웹 서버, 동시 처리

```
# invoke - 단일 호출
result = chain.invoke({"topic": "AI"})

# batch - 여러 입력 동시 처리
results = chain.batch([
    {"topic": "AI"},
    {"topic": "ML"},
    {"topic": "DL"}
])

# stream - 실시간 출력
for chunk in chain.stream({"topic": "AI"}):
    print(chunk, end="", flush=True)
```

LLM Chain(Prompt+LLM)

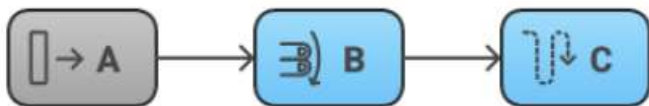
■ Multi Chain

- 여러 개의 체인을 연결하거나 병렬로 실행하여 복잡한 워크플로우를 구성하는 패턴
- LCEL(LangChain Expression Language)을 사용하면 체인을 파이프(|)와 RunnableParallel로 유연하게 조합할 수 있습니다.
- 체인 간 데이터 전달 : 출력을 다음 체인의 입력으로
- 순차적 체인(Sequential) 연결 : 파이프 연산자로 체인 연결
단계별 처리
예] 번역 → 요약 → 검토
- 병렬 체인 실행 : RunnableParallel로 동시 실행
독립적 작업 동시 실행
예] 여러 관점에서 분석
- 조건부 분기 : 입력에 따른 체인 선택
입력에 따라 다른 처리
언어별 다른 프롬프트

LLM Chain(Prompt+LLM)

■ Sequential Chain

```
chain = A | B | C
```



```
from langchain.chat_models import init_chat_model
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

llm = init_chat_model("gpt-4o-mini")

# 1단계: 한국어를 영어로 번역
translate_prompt = ChatPromptTemplate.from_template(
    "다음 한국어를 영어로 번역하세요: {korean_word}"
)

# 2단계: 영어 단어 설명
explain_prompt = ChatPromptTemplate.from_template(
    "다음 영어 단어를 한국어로 자세히 설명하세요: {english_word}"
)
```

LLM Chain(Prompt+LLM)

■ Sequential Chain

```
# 체인 1: 번역
chain1 = translate_prompt | llm | StrOutputParser()

# 체인 2: 번역 결과를 입력으로 사용
chain2 = (
    {"english_word": chain1}
    | explain_prompt
    | llm
    | StrOutputParser()
)

# 실행
result = chain2.invoke({"korean_word": "인공지능"})
print(result)
```

LLM Chain(Prompt+LLM)

■ 다단계 Sequential Chain

- **RunnablePassthrough** : 파이프라인(Runnable chain)을 이어주기 위한 Runnable
입력값을 소비하지 않고 다음 단계로 그대로 전달 동시에 다른 Runnable의 결과와 합성할 때 사용

```
from langchain.chat_models import init_chat_model
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnablePassthrough

llm = init_chat_model("gpt-4o-mini")

# 3단계 처리: 주제 분석 → 개요 작성 → 본문 작성
analyze_prompt = ChatPromptTemplate.from_template(
    "다음 주제의 핵심 키워드 3개를 추출하세요: {topic}"
)

outline_prompt = ChatPromptTemplate.from_template(
    """다음 키워드를 바탕으로 글의 개요를 작성하세요:
키워드: {keywords}
원본 주제: {topic}"""
)
```


LLM Chain(Prompt+LLM)

■ 다단계 Sequential Chain

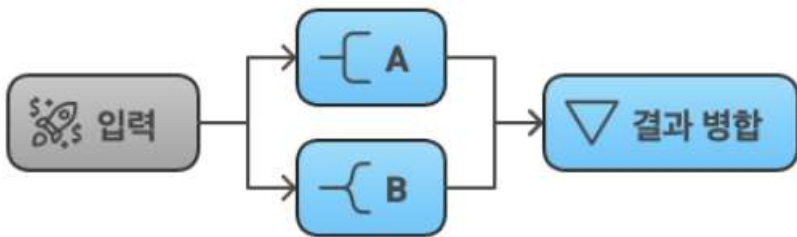
```
content_prompt = ChatPromptTemplate.from_template(
    """다음 개요를 바탕으로 300자 내외의 글을 작성하세요:
    개요: {outline}"""
)
# 체인 구성
chain = (
    {"topic": RunnablePassthrough()}
    | RunnablePassthrough.assign(
        keywords=analyze_prompt | llm | StrOutputParser()
    )
    | RunnablePassthrough.assign(
        outline=outline_prompt | llm | StrOutputParser()
    )
    | content_prompt
    | llm
    | StrOutputParser()
)

result = chain.invoke("기후 변화와 지속 가능한 발전")
print(result)
```

LLM Chain(Prompt+LLM)

■ Parallel Chain

```
chain = RunnableParallel(a=A, b=B)
```



```
from langchain.chat_models import init_chat_model
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnableParallel
```

```
llm = init_chat_model("gpt-4o-mini")
```

```
# 세 가지 관점에서 동시 분석
```

```
positive_prompt = ChatPromptTemplate.from_template(
    "{topic}의 긍정적인 측면 3가지를 설명하세요."
)
```

LLM Chain(Prompt+LLM)

■ Parallel Chain

```
negative_prompt = ChatPromptTemplate.from_template(
    "{topic}의 부정적인 측면 3가지를 설명하세요."
)
neutral_prompt = ChatPromptTemplate.from_template(
    "{topic}에 대한 객관적인 현황을 설명하세요."
)
# 병렬 체인 구성
parallel_chain = RunnableParallel(
    positive=positive_prompt | llm | StrOutputParser(),
    negative=negative_prompt | llm | StrOutputParser(),
    neutral=neutral_prompt | llm | StrOutputParser(),
)
# 실행 (세 체인이 동시에 실행됨)
results = parallel_chain.invoke({"topic": "원격 근무"})

print("=== 긍정적 측면 ===")
print(results["positive"])
print("\n=== 부정적 측면 ===")
print(results["negative"])
print("\n=== 객관적 현황 ===")
print(results["neutral"])
```

LLM Chain(Prompt+LLM)

■ Parallel Chain

- RunnableParallel(a=..., b=...) 독립 작업 동시 실행
- RunnablePassthrough.assign() 중간 결과 축적
- 각 체인에 `with_fallbacks()` 로 에러 처리
- 병렬 결과 통합

```
from langchain.chat_models import init_chat_model
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnableParallel, RunnablePassthrough

llm = init_chat_model("gpt-4o-mini")

# 병렬 분석
analysis_chain = RunnableParallel(
    pros=ChatPromptTemplate.from_template("{topic}의 장점") | llm | StrOutputParser(),
    cons=ChatPromptTemplate.from_template("{topic}의 단점") | llm | StrOutputParser(),
)
```

LLM Chain(Prompt+LLM)

■ Parallel Chain

```
# 결과 통합
synthesis_prompt = ChatPromptTemplate.from_template(
    """"다음 분석을 종합하여 결론을 작성하세요:

장점:
{pros}

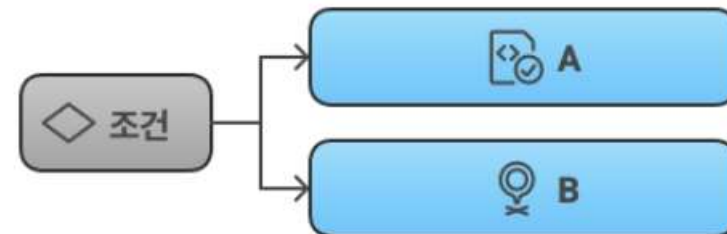
단점:
{cons}

균형 잡힌 결론을 3문장으로 작성하세요.""")
)
# 전체 체인: 병렬 분석 → 통합
full_chain = (
    analysis_chain
    | synthesis_prompt
    | llm
    | StrOutputParser()
)
result = full_chain.invoke({"topic": "인공지능"})
print(result)
```

LLM Chain(Prompt+LLM)

■ Branching Chain

```
chain = RunnableBranch(...)
```



```
from langchain.chat_models import init_chat_model
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnableBranch, RunnableLambda
```

```
llm = init_chat_model("gpt-4o-mini")
```

```
# 언어별 다른 프롬프트
```

```
korean_prompt = ChatPromptTemplate.from_template(
    "다음 한국어 질문에 한국어로 답변하세요: {question}"
)
```

```
english_prompt = ChatPromptTemplate.from_template(
    "Answer the following question in English: {question}"
)
```

LLM Chain(Prompt+LLM)

■ Branching Chain

```
default_prompt = ChatPromptTemplate.from_template(
    "Please answer: {question}"
)

# 언어 감지 함수
def detect_language(input_dict):
    question = input_dict.get("question", "")
    # 간단한 한글 감지
    if any('\uac00' <= char <= '\ud7a3' for char in question):
        return "korean"
    return "english"

# 조건부 분기
branch_chain = RunnableBranch(
    # (조건 함수, 실행할 체인) 튜플 목록
    (lambda x: detect_language(x) == "korean", korean_prompt | llm | StrOutputParser()),
    (lambda x: detect_language(x) == "english", english_prompt | llm | StrOutputParser()),
    # 기본값 (조건에 맞지 않을 때)
    default_prompt | llm | StrOutputParser()
)
```

LLM Chain(Prompt+LLM)

■ Branching Chain

```
# 테스트
result_kr = branch_chain.invoke({"question": "파이썬이란 무엇인가요?"})
result_en = branch_chain.invoke({"question": "What is Python?"})

print("한국어 질문 결과:", result_kr)
print("영어 질문 결과:", result_en)
```


LLM Chain(Prompt+LLM)

■ 함수 기반 라우팅

```
from langchain_core.runnables import RunnableLambda

def route_by_topic(input_dict):
    """주제에 따라 다른 체인 선택"""
    topic = input_dict.get("topic", "").lower()

    if "code" in topic or "programming" in topic:
        return "technical"
    elif "business" in topic or "strategy" in topic:
        return "business"
    else:
        return "general"

# 주제별 체인 정의
chains = {
    "technical": ChatPromptTemplate.from_template(
        "기술 전문가로서 답변: {question}"
    ) | llm | StrOutputParser(),
```

LLM Chain(Prompt+LLM)

■ 함수 기반 라우팅

```
"business": ChatPromptTemplate.from_template(
    "비즈니스 컨설턴트로서 답변: {question}"
) | llm | StrOutputParser(),

"general": ChatPromptTemplate.from_template(
    "일반적인 관점에서 답변: {question}"
) | llm | StrOutputParser(),
}

# 라우팅 체인
def routing_chain(input_dict):
    route = route_by_topic(input_dict)
    return chains[route].invoke(input_dict)

router = RunnableLambda(routing_chain)
# 테스트
result = router.invoke({
    "topic": "Python programming",
    "question": "효율적인 코드 작성 방법은?"
})
print(result)
```

LLM Chain(Prompt+LLM)

■ Multi-Chain

- RunnableLambda(func) : 데이터 변환, 전처리/후처리

```
from langchain.chat_models import init_chat_model
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnableParallel, RunnablePassthrough

llm = init_chat_model("gpt-4o-mini")

# 가상의 검색 함수
def retrieve_context(query: str) -> str:
    # 실제로는 벡터스토어 검색
    return f"검색된 컨텍스트: {query}에 대한 정보..."

# RAG 체인 구성
rag_chain = (
    # 1. 쿼리와 컨텍스트 병렬 준비
    RunnableParallel(
        question=RunnablePassthrough(),
        context=RunnableLambda(lambda x: retrieve_context(x["question"]))
    )
```

LLM Chain(Prompt+LLM)

■ Multi-Chain

```
# 2. 프롬프트 생성
| ChatPromptTemplate.from_template(
    """"컨텍스트를 참고하여 질문에 답변하세요.

컨텍스트: {context}

질문: {question}

답변:"""
    )
# 3. LLM 호출
| llm
# 4. 출력 파싱
| StrOutputParser()
)

result = rag_chain.invoke({"question": "LangChain이란?"})
print(result)
```

OutputParser

Output Parser

■ Output Parser

- LLM의 텍스트 출력을 구조화된 데이터로 변환하는 컴포넌트
- `langchain_core.output_parsers`
- 자연어 응답을 JSON, 리스트, Pydantic 객체 형식으로 포맷 변환하여 후속 처리를 용이하게 합니다
- `get_format_instructions()` : 모델에게 출력 형식 지시
- 출력 형식 유효성 자동 검증, 스키마 검증, 타입 체크
- `OutputFixingParser` : 파싱 실패 시 재시도, 오류 복구

Output Parser

■ Output Parser 클래스

Parser	출력 타입	특징 &용도
BaseOutputParser		모든 파서의 부모(추상) 클래스
StrOutputParser	str	LLM 출력을 단순 문자열로 변환
CommaSeparatedListOutputParser	List[str]	쉼표로 구분된 텍스트를 리스트로 변환
JsonOutputParser	dict	LLM 출력을 JSON 형식 파싱, 구조화된 데이터
PydanticOutputParser	Pydantic	LLM 출력을 Pydantic 모델 객체로 파싱
NumberedListOutputParser	list[str]	번호 매겨진 텍스트를 리스트로 변환
StructuredOutputParser	dict	여러 필드를 가진 구조화된 출력 파싱 (스키마 기반)
XMLOutputParser	dict	XML 형식 출력을 파싱
EnumOutputParser	Enum 멤버	정해진 enum 값 중 하나로 파싱
DatetimeOutputParser	datetime	날짜/시간 문자열 파싱
OutputFixingParser	래핑한 파서	파싱 실패 시 LLM을 다시 호출해 형식 교정
RetryOutputParser	래핑한 파서	파싱 실패 시 원본 프롬프트까지 활용해 재시도

Output Parser

■ Output Parser 주요 메서드

메서드	특징 &용도
parse(text: str)	LLM이 생성한 문자열을 받아 원하는 형식으로 변환
parse_result(result: LLMResult)	LLMResult(여러 generation 포함) 객체를 받아 첫 번째 결과를 parse()로 처리
invoke(input, config=None)	Runnable 인터페이스 체인 메서드
ainvoke(input, config=None)	invoke의 비동기 버전
batch(inputs)	여러 입력을 한번에 처리
stream(input)	스트리밍 출력 처리 (지원하는 파서의 경우 토큰 단위로 점진적 파싱)
get_format_instructions()	LLM에게 "이 형식으로 답해줘"라고 지시하는 프롬프트 문구 생성 (주로 구조화 파서에서 사용)
OutputParserException	파싱 실패 시 발생시키는 표준 예외 클래스

Output Parser

■ StrOutput Parser

- 가장 기본적인 파서로, AIMessage에서 텍스트 내용만 추출

```
from langchain_core.output_parsers import StrOutputParser, JsonOutputParser
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI

model = ChatOpenAI()
prompt = ChatPromptTemplate.from_template("{question}에 대해 답해줘")
parser = StrOutputParser()

chain = prompt | model | parser
result = chain.invoke({"question": "LangChain이 뭐야"})
print(result) # 순수 문자열
```

Output Parser

■ PydanticOutputParser

```
from langchain_core.output_parsers import PydanticOutputParser
from pydantic import BaseModel, Field

class Answer(BaseModel):
    topic: str = Field(description="주제")
    summary: str = Field(description="요약")

parser = PydanticOutputParser(pydantic_object=Answer)

# 프롬프트에 형식 지시문 자동 삽입
format_instructions = parser.get_format_instructions()

prompt = ChatPromptTemplate.from_template(
    "{question}\n\n{format_instructions}"
).partial(format_instructions=format_instructions)

chain = prompt | model | parser
result = chain.invoke({"question": "LangChain을 한 줄로 설명해줘"})
print(result.topic, result.summary)
```

Output Parser

■ CommaSeparatedListOutputParser

- 쉼표로 구분된 항목을 리스트로 변환

```
from langchain_core.output_parsers import CommaSeparatedListOutputParser
from langchain_core.prompts import PromptTemplate
from langchain.chat_models import init_chat_model

output_parser = CommaSeparatedListOutputParser()
# 포맷 지시사항 확인
format_instructions = output_parser.get_format_instructions()
print(format_instructions)
# Your response should be a list of comma separated values, eg: `foo, bar, baz`
# 프롬프트에 포맷 지시사항 포함
prompt = PromptTemplate(
    template="5가지 {subject}을(를) 나열하세요.\n{format_instructions}",
    input_variables=["subject"],
    partial_variables={"format_instructions": format_instructions},
)
llm = init_chat_model("gpt-4o-mini", temperature=0)
chain = prompt | llm | output_parser
result = chain.invoke({"subject": "인기 있는 프로그래밍 언어"})
print(result)
print(type(result))
```

Output Parser

■ JsonOutputParser - JSON 파싱

- Pydantic 모델을 사용하여 JSON 출력을 구조화

```
from langchain_core.output_parsers import JsonOutputParser
from langchain_core.prompts import PromptTemplate
from langchain.chat_models import init_chat_model
from pydantic import BaseModel, Field
```

```
# Pydantic 모델 정의
```

```
class BookInfo(BaseModel):
    title: str = Field(description="책 제목")
    author: str = Field(description="저자 이름")
    year: int = Field(description="출판 연도")
    genre: str = Field(description="장르")
```

```
# JSON 파서 생성
```

```
parser = JsonOutputParser(pydantic_object=BookInfo)
```

```
# 포맷 지시사항 확인
```

```
print(parser.get_format_instructions())
```

Output Parser

■ JsonOutputParser - JSON 파싱

```
# 프롬프트 구성
prompt = PromptTemplate(
    template="다음 책에 대한 정보를 제공해주세요: {book_name}\n{format_instructions}",
    input_variables=["book_name"],
    partial_variables={"format_instructions": parser.get_format_instructions()},
)

# 체인 구성
llm = init_chat_model("gpt-4o-mini", temperature=0)
chain = prompt | llm | parser

# 실행
result = chain.invoke({"book_name": "해리포터와 마법사의 돌"})
print(result)
```

Output Parser

■ with_structured_output

- 모델의 네이티브 JSON 스키마 기능을 활용하여 구조화 출력

```
from langchain.chat_models import init_chat_model
from pydantic import BaseModel, Field
# 출력 스키마 정의
class MovieReview(BaseModel):
    """영화 리뷰 정보"""
    title: str = Field(description="영화 제목")
    rating: int = Field(description="1-10점 평점")
    pros: list[str] = Field(description="장점 목록")
    cons: list[str] = Field(description="단점 목록")
    summary: str = Field(description="한 줄 요약")
# 구조화 출력 설정
llm = init_chat_model("gpt-4o-mini")
structured_llm = llm.with_structured_output(MovieReview)
# 실행
review = structured_llm.invoke("인셉션 영화를 리뷰해주세요.")
print(f"제목: {review.title}")
print(f"평점: {review.rating}/10")
print(f"장점: {review.pros}")
print(f"단점: {review.cons}")
print(f"요약: {review.summary}")
```

Output Parser

■ TypedDict

- Pydantic 없이 간단하게 구조화
- 타입 검증 불필요

```
from typing import TypedDict
from langchain.chat_models import init_chat_model

# TypedDict로 스키마 정의
class WeatherInfo(TypedDict):
    city: str
    temperature: int
    condition: str
    humidity: int

# 구조화 출력 설정
llm = init_chat_model("gpt-4o-mini")
structured_llm = llm.with_structured_output(WeatherInfo)

# 실행
weather = structured_llm.invoke("서울의 현재 날씨를 알려주세요.")
print(weather)
```

Output Parser

■ include_raw 옵션

- 원본 AIMessage와 파싱된 결과를 함께 받음

```
from langchain.chat_models import init_chat_model
from pydantic import BaseModel, Field

class Summary(BaseModel):
    main_point: str = Field(description="핵심 요약")
    keywords: list[str] = Field(description="키워드 목록")

llm = init_chat_model("gpt-4o-mini")
structured_llm = llm.with_structured_output(Summary, include_raw=True)

result = structured_llm.invoke("인공지능의 발전에 대해 요약해주세요.")

print("파싱된 결과:", result["parsed"])
print("원본 메시지:", result["raw"])
print("파싱 오류:", result["parsing_error"])
```


Memory와 대화 관리

메모리와 대화 관리

■ 메모리의 역할 & 유형

- 대화 기록 저장하고 관리
- 대화의 흐름과 맥락(컨텍스트) 보존
- 사용자별 대화 세션 관리(개인화)
- 토큰 사용량 최적화를 위한 메모리 관리
- 단기 메모리 (Short-term Memory) : 현재 대화 세션 동안만 유지되는 메모리
 - 전체 히스토리 - 모든 메시지를 그대로 저장, 짧은 대화, 정확한 맥락 필요
 - 윈도우 메모리 - 최근 N개 메시지만 유지, 일반적인 챗봇에 사용
 - 요약 메모리 - 이전 대화를 요약하여 저장, 긴 대화, 비용 최적화에 사용
 - 토큰 버퍼 - 토큰 수 기준으로 자동 트리밍, 토큰 예산 관리
- 장기 메모리 (Long-term Memory) : 여러 세션에 걸쳐 유지되는 영구적 메모리
 - 사용자 프로필 - 사용자 선호도, 속성 저장, 개인화된 응답에 사용
 - 대화 요약 - 과거 세션 요약 저장, 세션 간 맥락 유지에 사용
 - 지식 추출 - 대화에서 핵심 정보 추출, 장기 학습 에이전트에 사용

메모리와 대화 관리

■ 메모리의 역할 & 유형

- 간단한 챗봇 : [RunnableWithMessageHistory](#) 사용
- 복잡한 에이전트 : [LangGraph checkpointer](#) 사용
- [MessagesPlaceholder\(variable_name="history"\)](#) : 대화 히스토리가 삽입될 위치를 지정
- Placeholder를 통해 이전 대화 내용이 프롬프트에 동적으로 포함됩니다
- session_id를 통해 서로 다른 사용자나 대화 세션을 구분합니다
- 프로세스가 종료되면 데이터가 사라지는 인메모리 방식입니다

파라미터	패키지
InMemoryChatMessageHistory	메모리 내에서 대화 기록을 저장, 재시작 시 소멸, 개발/테스트에 적합
FileChatMessageHistory	파일 기반, 간단한 영구 저장, 단일 서버, 소규모 적용에 적합
SQLChatMessageHistory	DB 저장, SQL 지원, 관계형 DB 환경 적용에 적합
RedisChatMessageHistory	빠른 읽기/쓰기, TTL 지원, 분산 환경, 캐싱 적용에 적합
PostgresChatMessageHistory	안정적, 트랜잭션 지원, 프로덕션/영구 저장 적용에 적합

메모리와 대화 관리

■ RunnableWithMessageHistory

- 이전 모든 대화 내용이 자동으로 현재 프롬프트에 포함되어 연속적인 대화가 가능
- 서로 다른 session_id를 사용하여 여러 사용자나 대화 스레드를 독립적으로 관리할 수 있습니다.
- 복잡한 메모리 관리 로직 없이도 대화형 AI를 구현할 수 있습니다.

메모리와 대화 관리

■ RunnableWithMessageHistory

```
from langchain_anthropic import ChatAnthropic
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder
from langchain_core.runnables.history import RunnableWithMessageHistory
from langchain_core.chat_history import InMemoryChatMessageHistory

# 1. 세션별 메모리 저장소
store = {}
def get_session_history(session_id: str):          # 세션별 히스토리를 관리하는 함수
    if session_id not in store:
        store[session_id] = InMemoryChatMessageHistory()
    return store[session_id]
# 2. 모델 설정
llm = ChatAnthropic(
    model="claude-haiku-4-5-20251001",
    temperature=0,
    max_tokens=500
)
# 3. 프롬프트 템플릿
prompt = ChatPromptTemplate.from_messages([
    ("system", "당신은 친절한 AI 어시스턴트입니다."),
    MessagesPlaceholder(variable_name="history"),
    ("human", "{question}")
])
```

메모리와 대화 관리

■ RunnableWithMessageHistory

```
# 4. 체인 생성
chain = prompt | llm

# 5. 메모리가 연결된 체인
chain_with_memory = RunnableWithMessageHistory(
    chain,
    get_session_history,          # 세션 히스토리를 가져오는 함수
    input_messages_key="question", # 사용자 입력 키
    history_messages_key="history"  # 대화 기록 키
)

# 6. 대화 실행
config = {"configurable": {"session_id": "user_123"}}
# 첫 번째 대화
response1 = chain_with_memory.invoke(
    {"question": "안녕하세요"},
    config=config
)
print("AI:", response1.content)
print("-" * 50)
)
```

메모리와 대화 관리

■ RunnableWithMessageHistory

```
# 두 번째 대화 (이전 대화 기억)
response2 = chain_with_memory.invoke(
    {"question": "내 이름은 김철수입니다."},
    config=config
)
print("AI:", response2.content)
print("-" * 50)

# 세 번째 대화 (이름 기억 확인)
response3 = chain_with_memory.invoke(
    {"question": "내 이름이 뭐였죠?"},
    config=config
)
print("AI:", response3.content)
```

메모리와 대화 관리

■ LangGraph 기반 메모리

- 데이터베이스 기반 저장소를 사용

```
from langchain_community.chat_message_histories import RedisChatMessageHistory

def get_session_history(session_id: str):
    return RedisChatMessageHistory(session_id, url="redis://localhost:6379")
```

- 토큰 제한 관리

```
from langchain_core.messages import trim_messages

def get_trimmed_history(session_id: str):
    history = get_session_history(session_id)
    # 최대 10개 메시지만 유지
    trimmed = trim_messages(
        history.messages,
        max_tokens=4000,
        strategy="last"
    )
    return trimmed
```


메모리와 대화 관리

■ FileChatMessageHistory

- 파일 기반 저장소 : 동시 접근 시 파일 잠금 문제가 발생할 수 있습니다.
단일 서버 환경에서만 사용을 권장

```
# 대화 히스토리를 JSON 파일로 저장
from langchain_community.chat_message_histories import FileChatMessageHistory

def get_file_session_history(session_id: str):
    return FileChatMessageHistory(f"chat_histories/{session_id}.json") #chat_histories/ 디렉토리가 미리 생성

chain_with_file_history = RunnableWithMessageHistory(
    chain,
    get_file_session_history,
    input_messages_key="question",
    history_messages_key="history",
)
```

메모리와 대화 관리

■ SQLChatMessageHistory

```
#SQL 기반 저장소로, 로컬 파일 DB를 사용
from langchain_community.chat_message_histories import SQLChatMessageHistory

def get_sql_session_history(session_id: str):
    return SQLChatMessageHistory(
        session_id=session_id,
        connection_string="sqlite:///chat_history.db"
    )

chain_with_sql_history = RunnableWithMessageHistory(
    chain,
    get_sql_session_history,
    input_messages_key="question",
    history_messages_key="history",
)
```

메모리와 대화 관리

■ RedisChatMessageHistory

```
pip install redis
```

```
from langchain_community.chat_message_histories import RedisChatMessageHistory

def get_redis_session_history(session_id: str):
    return RedisChatMessageHistory(
        session_id=session_id,
        url="redis://localhost:6379",
        ttl=3600 # 메시지 만료 시간, 1시간 후 자동 삭제
    )

chain_with_redis_history = RunnableWithMessageHistory(
    chain,
    get_redis_session_history,
    input_messages_key="question",
    history_messages_key="history",
)
```

메모리와 대화 관리

■ PostgresChatMessageHistory

```
pip install psycopg2-binary
```

```
from langchain_community.chat_message_histories import PostgresChatMessageHistory

def get_postgres_session_history(session_id: str):
    return PostgresChatMessageHistory(
        session_id=session_id,
        connection_string="postgresql://user:password@localhost:5432/chatdb"
    )

chain_with_postgres_history = RunnableWithMessageHistory(
    chain,
    get_postgres_session_history,
    input_messages_key="question",
    history_messages_key="history",
)
```

메모리와 대화 관리

■ LangGraph 기반 메모리

```
from langgraph.checkpoint.memory import InMemorySaver
from langchain.agents import create_agent

# 체크포인트로 상태 저장
checkpointer = InMemorySaver()

agent_with_memory = create_agent(
    model="openai:gpt-4o-mini",
    tools=[],
    checkpointer=checkpointer # 자동 상태 저장
)

# thread_id로 대화 구분
config = {"configurable": {"thread_id": "conversation_1"}}
# 첫 번째 대화
response1 = agent_with_memory.invoke(
    {"messages":[{"role": "user", "content": "안녕하세요"}]},
    config=config
)
print("AI:", response1['messages'][-1].content)
print("-" * 50)
```

메모리와 대화 관리

■ LangGraph 기반 메모리

```
# 두 번째 대화 (이전 대화 기억)
response2 = agent_with_memory.invoke(
    {"messages":[{"role": "user", "content": "내 이름은 김철수입니다."}]},
    config=config
)
print("AI:", response2['messages'][-1].content)
print("-" * 50)

# 세 번째 대화 (이름 기억 확인)
response3 = agent_with_memory.invoke(
    {"messages":[{"role": "user", "content": "내 이름이 뭐였죠?"}]},
    config=config
)
print("AI:", response3['messages'][-1].content)
```

메모리와 대화 관리

■ 단기 메모리(Short-term Memory)

- 단일 대화 스레드 내에서 이전 상호작용을 기억하는 시스템
- 이전 대화를 기억하고, 피드백을 학습하며, 사용자 선호도에 적응할 수 있게 해줍니다.
- 스레드(Thread)는 하나의 세션에서 여러 상호작용을 조직화하는 단위
- 에이전트는 시스템 메시지(지침)와 사용자 메시지(입력)를 포함한 메시지 목록을 유지합니다.
- 대화가 길어질수록 컨텍스트 윈도우 제한에 도달할 수 있으므로, 오래된 정보를 제거하거나 "잊어버리는" 기법이 필요합니다.
- LangChain 1.0 에이전트에서 단기 메모리를 사용하려면 `create_agent`에 `checkpoint`를 전달합니다

메모리와 대화 관리

■ 단기 메모리(Short-term Memory)

```
from langchain.agents import create_agent
from langchain.chat_models import init_chat_model
from langgraph.checkpoint.memory import MemorySaver

# Checkpointer 생성 (메모리 기반)
checkpointer = MemorySaver()

# 에이전트 생성 - checkpointer를 직접 전달
model = init_chat_model("openai:gpt-4o")
agent = create_agent(
    model,
    tools=[],
    checkpointer=checkpointer # 메모리 활성화
)
# 대화 실행 - agent.invoke() 직접 호출
config = {"configurable": {"thread_id": "conversation-1"}}

result1 = agent.invoke(
    {"messages": [("user", "안녕하세요, 제 이름은 철수입니다.")]},
    config=config
)
```


메모리와 대화 관리

■ 단기 메모리(Short-term Memory)

```
result2 = agent.invoke(
    {"messages": [("user", "제 이름이 뭐죠?)]},
    config=config
)
print(result2["messages"][-1].content)
# 출력: "당신의 이름은 철수입니다."
```

- 프로덕션에서는 데이터베이스 기반 checkpointer를 사용합니다.

```
from langgraph.checkpoint.postgres import PostgresSaver

# PostgreSQL checkpointer
checkpointer = PostgresSaver.from_conn_string(
    "postgresql://user:pass@localhost/dbname"
)

agent = create_agent(
    model,
    tools=[],
    checkpointer=checkpointer
)
```

메모리와 대화 관리

■ Short-term Memory

- LLM의 컨텍스트 윈도우는 제한되어 있습니다.
- 긴 대화에서는 메시지를 트리밍하여 최신 N개의 메시지나 토큰만 유지할 수 있습니다.

```
from langchain.agents import create_agent
from langchain.agents.middleware import before_model
from langchain.chat_models import init_chat_model
from langchain_core.messages import trim_messages
from langgraph.checkpoint.memory import MemorySaver

@before_model
def trim_message_history(state: dict) -> dict:
    """모델 호출 전에 메시지를 트리밍합니다."""
    messages = state.get("messages", [])

    # 최대 4096 토큰만 유지
    trimmed = trim_messages(
        messages,
        max_tokens=4096,
        strategy="last", # 마지막 메시지부터 유지
        token_counter=len # 실제로는 토큰 카운터 함수 사용
    )
    return {"messages": trimmed}
```

메모리와 대화 관리

■ Short-term Memory

```
# 에이전트에 미들웨어 적용
model = init_chat_model("openai:gpt-4o")
agent = create_agent(
    model,
    tools=[],
    checkpoint=MemorySaver(),
    middleware=[trim_message_history]
)
```

메모리와 대화 관리

■ Message Deletion

- 특정 메시지를 상태에서 영구적으로 삭제할 수 있습니다.
- 민감한 정보를 제거하거나 불필요한 메시지를 정리할 때 유용합니다.
- 정보 손실을 초래할 수 있습니다

```
from langgraph.graph import RemoveMessage

def delete_old_messages(state: dict) -> dict:
    """5개보다 오래된 메시지를 삭제합니다."""
    messages = state.get("messages", [])

    if len(messages) > 5:
        # 삭제할 메시지 ID 수집
        messages_to_delete = messages[:-5]
        remove_commands = [
            RemoveMessage(id=msg.id) for msg in messages_to_delete
        ]
        return {"messages": remove_commands}

    return {}
```

메모리와 대화 관리

■ Message Trimming

- 모든 메시지 삭제

```
def clear_all_messages(state: dict) -> dict:
    """대화 기록을 완전히 초기화합니다."""
    messages = state.get("messages", [])

    # 모든 메시지 삭제
    remove_commands = [RemoveMessage(id=msg.id) for msg in messages]
    return {"messages": remove_commands}
```

메모리와 대화 관리

■ Message Trimming

- 민감 정보 삭제

```
from langchain_core.messages import HumanMessage, AIMessage

def remove_sensitive_info(state: dict) -> dict:
    """비밀번호나 개인정보가 포함된 메시지를 삭제합니다."""
    messages = state.get("messages", [])
    sensitive_keywords = ["password", "비밀번호", "주민번호"]

    to_remove = []
    for msg in messages:
        if any(keyword in msg.content.lower() for keyword in sensitive_keywords):
            to_remove.append(RemoveMessage(id=msg.id))

    return {"messages": to_remove}
```

메모리와 대화 관리

■ Message Summarization

- 오래된 메시지를 요약하여 중요한 정보를 유지하면서 컨텍스트를 줄입니다.

```
from langchain.agents import create_agent
from langchain.agents.middleware import SummarizationMiddleware
from langchain.chat_models import init_chat_model
from langgraph.checkpoint.memory import MemorySaver
# 요약용 모델 (저렴한 모델 사용)
summarizer = init_chat_model("openai:gpt-4o-mini")
# 요약 미들웨어 생성
summarization = SummarizationMiddleware(
    summarizer=summarizer,
    max_messages=10, # 10개 이상이면 요약
    summary_prompt="이전 대화를 간단히 요약해주세요."
)
# 에이전트에 적용
model = init_chat_model("openai:gpt-4o")
agent = create_agent(
    model,
    tools=[],
    checkpointer=MemorySaver(),
    middleware=[summarization]
)
```

메모리와 대화 관리

■ Message Summarization

- 특정 메시지 타입만 유지하거나, 중요도에 따라 선별적으로 삭제하는 커스텀 전략도 구현할 수 있습니다.

```
# 중요 메시지만 유지
```

```
@before_model
```

```
def keep_important_messages(state: dict) -> dict:
```

```
    """중요 태그가 있는 메시지만 유지합니다."""
```

```
    messages = state.get("messages", [])
```

```
# 최근 5개 + 중요 메시지 유지
```

```
important = [m for m in messages if "important" in m.metadata.get("tags", [])]
```

```
recent = messages[-5:]
```

```
# 중복 제거
```

```
kept = {m.id: m for m in important + recent}.values()
```

```
return {"messages": list(kept)}
```


메모리와 대화 관리

■ Message Summarization

```
# 시간 기반 삭제

from datetime import datetime, timedelta

@before_model
def delete_old_by_time(state: dict) -> dict:
    """24시간 이상 된 메시지를 삭제합니다."""
    messages = state.get("messages", [])
    cutoff = datetime.now() - timedelta(hours=24)

    to_remove = []
    for msg in messages:
        msg_time = msg.metadata.get("timestamp")
        if msg_time and msg_time < cutoff:
            to_remove.append(RemoveMessage(id=msg.id))

    return {"messages": to_remove}
```

메모리와 대화 관리

■ Memory Store 구조

- LangGraph는 장기 메모리를 JSON 문서 형태로 Store에 저장
- 장기 메모리를 사용하려면 먼저 Store를 생성해야 합니다.
- 네임스페이스 (Namespace) : 더처럼 메모리를 계층적으로 구분하는 역할을 합니다.
- 키 (Key) : 파일 이름처럼 특정 메모리를 식별하는 고유 값

```
# 메모리 기반 Store  
store = InMemoryStore()
```

```
from langgraph.store.postgres import PostgresStore  
  
# PostgreSQL 기반 Store  
store = PostgresStore.from_conn_string(  
    "postgresql://user:pass@localhost/dbname"  
)
```

메모리와 대화 관리

■ Memory Store 구조

- 에이전트와 연동

```
# 에이전트와 연동

from langchain.agents import create_agent
from langchain.chat_models import init_chat_model
from langgraph.checkpoint.memory import MemorySaver

model = init_chat_model("openai:gpt-4o")

# 에이전트 생성 - store와 checkpointer를 직접 전달
agent = create_agent(
    model,
    tools=[],
    checkpointer=MemorySaver(), # 단기 메모리
    store=store # 장기 메모리 Store
)
```